EVO-BENCH: CAN LANGUAGE MODELS IMPROVE AGENT HARNESS?

Lisheng Huang[1]*, **Chen Yang**[2]*, **Hao Zhou**[2], **Huatong Song**[1],
Zongchao Chen[2], **Ran Le**[2], **Yang Song**[2]†, **Wayne Xin Zhao**[1]†, **Tao Zhang**[2]
[1]Gaoling School of Artificial Intelligence, Renmin University of China
[2]BOSS Zhipin, Beijing, China
huanglisheng@ruc.edu.cn, batmanfly@gmail.com, songyang@kanzhun.com

Leaderboard Code Dataset

ABSTRACT

Large Language Models (LLMs) have driven rapid progress in autonomous agents, yet standard evaluations remain confined to static task solving. An emerging frontier is harness evolution—the agent's capacity to autonomously optimize its own operating harness. However, systematically benchmarking this capability remains challenging, as existing evaluations fail to isolate harness improvements from base model strength, prevent task-specific overfitting, or capture long-horizon iterative research. To address these challenges, we introduce **Evo-Bench**, the first benchmark designed to evaluate models' intrinsic harness-evolving capabilities across Search, Office, and General agent domains. To rigorously isolate this capability, Evo-Bench employs a novel harness-guided construction framework: it leverages auxiliary-task evolution to identify tasks genuinely sensitive to framework improvements, followed by sensitivity-aware stratified splitting to ensure robust cross-suite generalization. Extensive evaluations across nine frontier and open-weight models reveal that top models achieve massive absolute gains reaching 16.6 points, closely approaching state-of-the-art human-engineered baselines. Crucially, while autonomous evolution outpeforms artificial harness in General tasks and excels in Search tasks, it struggles in Office tasks that demand highly specific processing workflows. Furthermore, our analysis exposes critical temporal anomalies like early saturation, while demonstrating that the synthesized harnesses act as highly transferable reasoning structures, consistently boosting diverse policy models.

1 INTRODUCTION

Recent breakthroughs in large language models (LLMs) have remarkably elevated the capacity of agentic systems to execute complex, long-horizon tasks at an end-to-end, system-level scale DeepSeek-AI (2026); Zeng et al. (2026); Anthropic (2026b;a). These gains arise not only from improvements in the underlying models, but also from carefully engineered agent harnesses that structure, coordinate, and constrain agent behavior, as exemplified by Claude Code Anthropic (2025) and Codex OpenAI (2025). In this context, an increasing body of work therefore seeks to explore the potential for increasingly capable agentic systems to achieve self-improvement, including autonomous scientific research Novikov et al. (2025), automated model post-training Rank et al. (2026), and autonomous harness evolution Zhang et al. (2026b); Lee et al. (2026); Chen et al. (2026). Among these lines of research, harness evolution is generally considered the first step toward achieving self-improvement Weng (2026). Consequently, a pivotal question emerges for the foundational models: *can language models truly improve agent harnesses, and how can we systematically benchmark this evolutionary capability?*

Recent work has explored harness evolution on two key fronts: methodologically, progressing from prompt search to end-to-end executable scaffold refactoring; and evaluatively, benchmarking evolu-

*Equal contribution.
†Corresponding authors.

tion methods Jiang et al. (2026); Zheng et al. (2026) and agent development Lu et al. (2026) while analyzing underlying harness benefits Jiang et al. (2026); Zheng et al. (2026). Collectively, these studies demonstrate how harnesses evolve and what constitutes success. However, systematically benchmarking an LLM's intrinsic capacity for harness evolution introduces unique, unaddressed obstacles.

Specifically, we identify three key challenges. First, *harness sensitivity*: benchmark task performance must be responsive to harness improvements rather than dominated by base model strength. Second, *cross-split generalization*: validation and evaluation splits must exhibit aligned responsiveness to prevent task-specific overfitting (Wang et al., 2026). Third, *long-horizon evolution*: models must sustain multi-round iterative refinement—diagnosing failures, formulating hypotheses, and progressively updating code. Unaddressed, these challenges conflate true harness-evolving capability with model strength or task-specific overfitting.

To address these challenges, we introduce **Evo-Bench**, the first benchmark evaluating models' intrinsic harness-evolving capability. Evo-Bench adopts a controlled, long-horizon research setting with a fixed policy model across three domains (Search, Office, and General Agent tasks) over five established benchmarks Wei et al. (2025); Hendrycks et al. (2026); Patwardhan et al. (2025); Vidgen et al. (2026); Ye et al. (2026). To rigorously construct this benchmark, we propose a novel *harness-guided benchmark construction framework*. Built upon this framework, Evo-Bench incorporates three core designs: (1) *harness-sensitive task construction*, identifying tasks responsive to framework improvements; (2) *sensitivity-aware stratified splitting*, ensuring strict distributional consistency and disjoint validation and evaluation suites; and (3) *controlled long-horizon evolution*, enabling sustained optimization of agent harnesses.

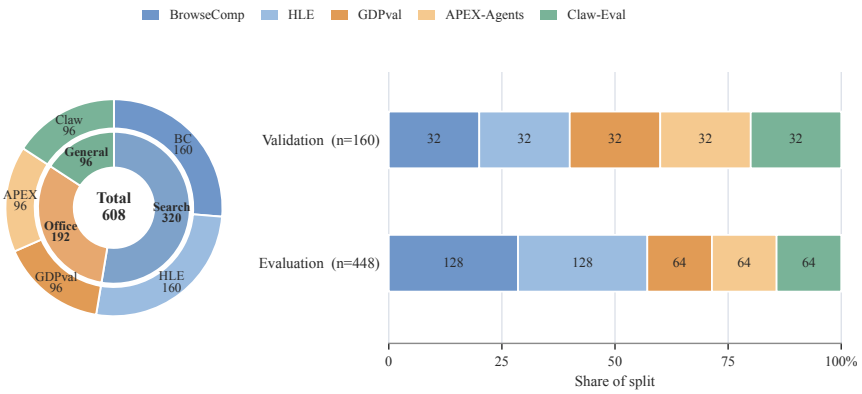
In our main evaluation across 9 frontier and open-weight models on **Evo-Bench**, top evolvers like GPT-5.6 Sol and Claude Opus 4.8 achieve substantial gains of 16.6 and 16.1 over the seed harness, respectively, closely approaching the human-engineered baseline of 47.5. Our analysis yields three key insights: (1) *evolutionary behavior exhibits early saturation*, with models rapidly discovering high-quality structures before introducing detrimental modifications in later rounds; (2) *gains are highly domain-dependent*, as evolvers effectively replicate web navigation in Search for massive gains and can even surpass manual engineering in General tasks, but struggle in Office tasks that demand highly specific processing workflows; and (3) *synthesized harnesses act as transferable reasoning structures*, demonstrating cross-policy robustness by driving consistent gains across diverse policy models from Qwen, DeepSeek, and GLM.

To summarize, our contributions are three-fold:

- We introduce **Evo-Bench**, the first benchmark evaluating LLMs' intrinsic harness-evolving capability—their ability to autonomously refine executable code harnesses across diverse domains beyond static task solving.
- We propose a harness-guided benchmark construction framework that first induces diverse harnesses through auxiliary-task evolution and then constructs the final benchmark via harness-guided task selection.
- We provide a systematic scientific account of harness evolution across 9 frontier models, characterizing substantial gains reaching 16.6 points, temporal anomalies like early saturation, and the robust transferability of evolved structures across distinct policy architectures.

2 Related Work

Automated Harness Engineering. Recently, agentic self-improvement has evolved from prompt tuning to workflow optimization, and recently, to the automated refinement of the *executable harness code* that orchestrates reasoning, tool use, and memory Weng (2026). While early prompt-level Fernando et al. (2024); Agrawal et al. (2025); Zhang et al. (2025b) and workflow-level methods Hu et al. (2024); Zhang et al. (2025a) operate on constrained design spaces, recent harness-level engineering leverages diverse mechanisms. These include self-referential scaffold refinement Zelikman et al. (2024); Yin et al. (2024); Zhang et al. (2026b;a), evolutionary archive search over executable code Lee et al. (2026); Ursekar et al. (2026), and trajectory-guided optimization Chen et al. (2026), alongside recent joint harness–weight co-optimization Hebbar et al. (2026). Crucially, rather than



(a) Overall hierarchy (b) Validation vs. evaluation

Figure 1: Composition of Evo-Bench. The left panel summarizes the three-domain hierarchy and source-benchmark allocation; the right panel compares the validation and evaluation splits.

proposing a new optimization algorithm, our work provides a complementary orthogonal perspective: we fix the policy model and research protocol to systematically benchmark the intrinsic capacity of frontier foundation models to act as long-horizon harness engineers across heterogeneous domains.

Agent Benchmarks. Existing agent benchmarks predominantly measure task execution within static, fixed harnesses Jimenez et al. (2024); Xie et al. (2024); Yao et al. (2024); Wei et al. (2025); Patwardhan et al. (2025), whereas AI-R&D benchmarks evaluate capabilities in optimizing model weights or training pipelines Huang et al. (2023); Chan et al. (2025); Wijk et al. (2024); Starace et al. (2025); Rank et al. (2026). While the Meta-Agent Challenge evaluates the from-scratch generation of task-specific artifacts Lu et al. (2026), an emerging class of evaluations focuses on self-evolutionary dynamics, tracking stability Jiang et al. (2026), distinguishing generation from utilization Lin et al. (2026), assessing capability transfer Gao et al. (2026), or formalizing diagnostic simulators Zheng et al. (2026). Differing from these frameworks, **Evo-Bench** shifts the evaluation paradigm. Instead of evaluating task-specific artifacts or unconstrained self-evolution, we explicitly benchmark foundation models as long-horizon research agents tasked with evolving a single, shared general-purpose harness, validated under a rigorous, sensitivity-calibrated task split.

3 Evo-Bench

Evo-Bench is a benchmark for evaluating large language models' *harness-evolving capability*: the ability to conduct long-horizon, code-centric iterative improvement of an executable agent harness. Given a set of validation tasks, models are expected to diagnose failure patterns, formulate improvement hypotheses, and revise the harness implementation to achieve sustained performance gains.

Task Formulation. As shown in Figure 2, each Evo-Bench run consists of a fixed policy model π and an evolver E that iteratively improves the policy harness. At evolution iteration t, the policy agent is $A_t^{\text{task}} = (\pi, H_t)$, where the editable policy harness H_t governs how π interacts with target tasks. The evolver is $A^{\text{evo}} = (E, \mathcal{H}_{\text{evo}})$ and operates through a separate, fixed evolve harness $\mathcal{H}_{\text{evo}}$. Let $\mathcal{E}_t^{\text{val}} := \big((H_i, j_i^{\text{val}}, O_i^{\text{val}})\big)_{i<t}$ denote the cumulative validation-side evidence available at iteration t, where j_i^{val} is the aggregate validation score and O_i^{val} contains the task-level outcomes, policy trajectories, and diagnostic feedback from evaluation i. Starting from the common seed harness H_0, the evolver inspects the accumulated validation-side evidence, edits the current harness H_t, and requests a formal evaluation of the resulting revision on the visible validation suite at each iteration. The evolution process is constrained by a fixed resource budget $\mathbf{b} = (b^{\text{iter}}, b^{\text{time}}, b^{\text{steps}})$, which bounds the number of iterations, wall-clock time, and evolver steps. When the run terminates, the final revision H_T is frozen and evaluated on a disjoint held-out evaluation suite.

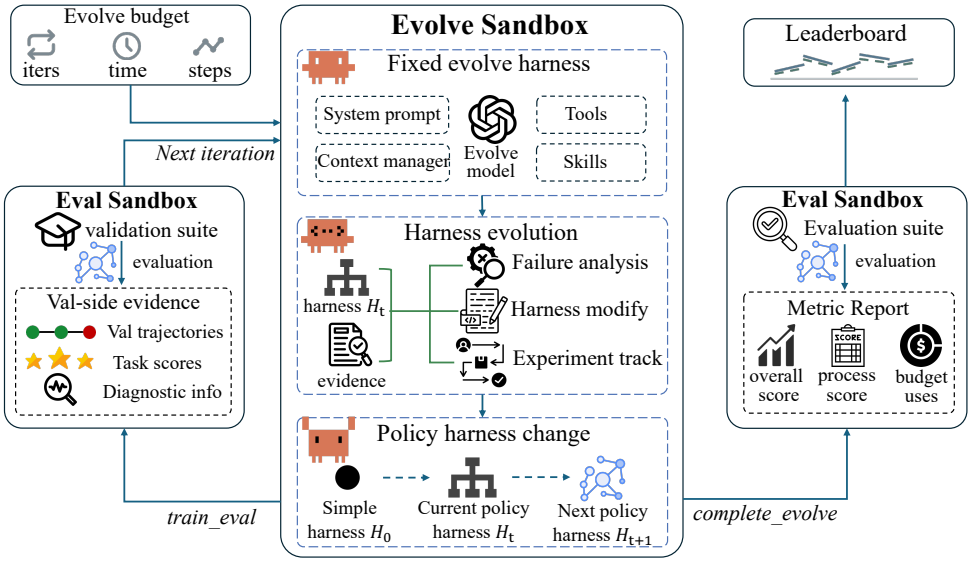


Figure 2: Overview of the Evo-Bench evaluation pipeline.

Task Domains and Composition. Evo-Bench covers three representative agent domains: search, office, and general agent tasks, evaluating harness evolution across diverse interaction patterns and tool-use requirements. To instantiate these domains, we select five established and challenging benchmarks: BrowseComp (Wei et al., 2025) and HLE (Hendrycks et al., 2026) for search, GDPval (Patwardhan et al., 2025) and APEX-Agents (Vidgen et al., 2026) for office, and Claw-Eval (Ye et al., 2026) for general agent tasks. Based on these benchmarks, we construct a 160-task visible validation suite $\mathcal{D}_{\text{val}}$ and a disjoint 448-task evaluation suite $\mathcal{D}_{\text{eval}}$. Each source benchmark contributes 32 validation tasks, while the evaluation suite contains 128 BrowseComp, 128 HLE, 64 GDPval, 64 APEX-Agents, and 64 Claw-Eval tasks. During evolution, the evolver can request validation evaluations only on $\mathcal{D}_{\text{val}}$, while $\mathcal{D}_{\text{eval}}$ is reserved for final evaluation after the harness is frozen. Figure 1 summarizes the domain hierarchy and the per-source allocation across the two splits.

Evolve Harness. The evolve harness operates as a fixed agent loop explicitly designed to support long-horizon autonomous work. Its core components, including the system prompt, execution tools, and context manager, are inspired by the design of Claude Code. Building upon this foundation, we integrate specialized skills for the evolution process, such as trajectory analysis and experiment tracking. We implement this custom harness natively within Evo-Bench to enable a controlled and transparent evaluation protocol.

Policy Harness. The policy harness is the evolving artifact optimized by the evolver. Its initial version H_0 is a minimal CodeAct Wang et al. (2024) loop with only a shell-execution tool and a final-answer completion tool. During validation evaluation on $\mathcal{D}_{\text{val}}$, the fixed policy model executes tasks in a separate sandbox through the current harness H_t. The evolver aims to transform this initial harness into a more capable, generalizable harness that boosts performance across all three domains.

Evaluation Metrics. We evaluate harness-evolving capability from two complementary perspectives: final generalization performance and evolutionary progress. For a given suite $\mathcal{D}$, let $\mathcal{S}(\pi, H; \mathcal{D})$ denote the aggregate score achieved by the policy model with harness H, computed from the source benchmarks' native scorer (defined in Table 1) and aggregated across target domains.

To measure the final performance of the evolved harness, we define **Overall Score** on the unseen evaluation suite:

$$\text{Overall}(E) := \mathcal{S}(\pi, H_T; \mathcal{D}_{\text{eval}}).$$

To measure the evolution progress, we define **Anytime Validation Score** as the average best-so-far validation performance over the evolution budget. At iteration t, the best validation score achieved up to and including iteration t is:

$$S_t^* = \max_{i \le t} \mathcal{S}(\pi, H_i; \mathcal{D}_{\text{val}}).$$

The Anytime Validation Score is then defined as:

$$\text{AnytimeVal}(E) := \frac{1}{b^{\text{iter}}} \sum_{t=1}^{b^{\text{iter}}} S_t^*.$$

If a run terminates early, its final best-so-far validation score is carried forward for the remaining budget.

Table 1: Statistics of the harness-guided task selection process. Tasks with non-positive sensitivity ($\text{Sens}(x) \le 0$) are filtered out prior to sensitivity-aware stratified selection. **Metrics**: $\text{Sens}(x)$ denotes harness sensitivity (Pearson correlation); $\text{Perf}(x)$ denotes average performance across the auxiliary harness set $\mathcal{H}_{\text{aux}}$. "LJ" represents LLM-as-a-Judge.

	Evaluation Protocol		**Candidate Filtering**		**Final Allocation**		$\text{Sens}(x)$		$\text{Perf}(x)$	
Source Dataset	Evaluator	Metric	Candidates	$\text{Sens}(x) \le 0$	Validation	Evaluation	Mean	Median	All	Selected
APEX-Agents	Rubric LJ	Pass@1	421	133	32	64	0.377	0.423	0.389	0.224
BrowseComp	LJ	Pass@1	768	47	32	128	0.256	0.236	0.444	0.271
Claw-Eval	LJ+Rule	Pass^3	157	60	32	64	0.386	0.317	0.839	0.747
GDPval	Rubric LJ	Mean	215	95	32	64	0.368	0.336	0.797	0.495
HLE	LJ	Pass@1	768	62	32	128	0.313	0.300	0.333	0.251

4 Benchmark Construction

To ensure that validation-side optimization reliably predicts held-out evaluation performance, benchmark tasks should satisfy two key properties: *Harness Sensitivity*, where task performance is sensitive to harness improvements while controlling for the underlying model, and *Cross-Suite Alignment*, where validation and evaluation suites exhibit consistent performance to harness variation. To this end, we design a Two-Stage Harness-Guided Benchmark Construction Framework. As shown in Figure 3, we first leverage auxiliary tasks to conduct controlled evolution experiments and generate a diverse set of auxiliary evolved harnesses, and then use these harnesses to characterize task-level harness sensitivity and difficulty for constructing the final benchmark task suites.

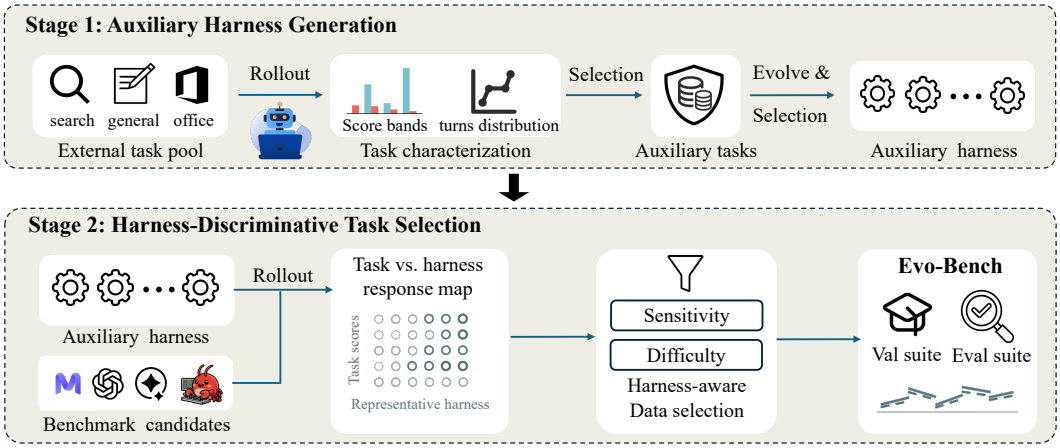


Figure 3: Overview of Two-Stage Harness-Guided Benchmark Construction Framework.

4.1 Stage 1: Auxiliary Harness Generation

Auxiliary Tasks Collection. We collect auxiliary tasks from diverse sources that are corpus- and instance-level disjoint from the five source benchmark datasets of Evo-Bench, including MiroRL Team & Team (2025), RedSearcher Chu et al. (2026), Auto-ClawEval Li et al. (2026), and internally constructed datasets. We then evaluate these tasks with DeepSeek-V4-Flash DeepSeek-AI (2026) through multiple independent rollouts and select tasks with lower average scores and longer interaction trajectories, which indicate greater room for harness improvements and stronger signals for harness evolution. This process results in a 320-task auxiliary set, comprising 128 search tasks, 128 office tasks, and 64 general agent tasks.

Auxiliary Harness Evolution. After constructing the auxiliary task set, we run independent evolution on it with multiple frontier models, collecting resulting harnesses throughout the process. We then deduplicate and select a representative subset to ensure sufficient diversity and coverage.

Specifically, we conduct four independent evolution experiments using four frontier models: GLM-5.2 Z.ai (2026), Claude-Opus-4.8 Anthropic (2026b), Claude-Sonnet-5 Anthropic (2026c), and GPT-5.6-Sol OpenAI (2026). Each experiment follows the same policy model, evolution protocol, and resource budget as the benchmark evaluation. We collect 73 formally evaluated harness variants and apply a deterministic diversity-aware selection procedure to identify 12 representative harnesses that maximize diversity across harness capabilities, tool orchestration, program structure, and auxiliary-task performance. We denote this selected set as $\mathcal{H}_{\text{aux}} = \{h_1, \ldots, h_K\}$, where $K = 12$.

4.2 Stage 2: Harness-Guided Task Selection

We evaluate candidate tasks under the selected harness set $\mathcal{H}_{\text{aux}}$ to identify tasks that reliably reflect harness improvements. Specifically, we evaluate 2,329 candidate tasks collected from APEX-Agents, BrowseComp, Claw-Eval, GDPval, and HLE using the 12 selected harnesses. We exclude multimodal tasks to maintain a unified interface and downsample BrowseComp and HLE to control evaluation cost while preserving their original task distributions.

For task x from the public benchmark $\mathcal{D}_{\text{public}}$, let $m_h(x)$ denote its score under harness $h \in \mathcal{H}_{\text{aux}}$. A straightforward criterion is to select tasks with high score variance across harnesses. However, high variance only captures the magnitude of performance differences and does not indicate whether these differences reflect the relative quality of harnesses. To more precisely measure whether a task consistently responds to harness quality, we compute the correlation between the task and overall harness quality, and combine this sensitivity measure with the average task score for task selection.

Harness Sensitivity and Task Difficulty. The score of an individual task is obtained directly from its execution result, while the overall quality of each harness is measured by its average performance across the remaining tasks. We compute the Pearson correlation coefficient between task-level scores and overall harness quality as the metric for task sensitivity. Specifically, we define two task-level metrics: harness sensitivity and average performance, as follows:

$$\mathrm{Sens}(x) = \mathrm{corr}\left(\{m_h(x)\}_{h\in\mathcal{H}_{\text{aux}}}, \{Q_h^{(-x)}\}_{h\in\mathcal{H}_{\text{aux}}}\right),$$
$$\mathrm{Perf}(x) = \frac{1}{|\mathcal{H}_{\text{aux}}|}\sum_{h\in\mathcal{H}_{\text{aux}}} m_h(x),$$

where $Q_h^{(-x)}$ denotes the leave-one-task-out average performance of harness h, computed over all tasks except x, and serves as a robust estimate of its overall quality.

Higher $\mathrm{Sens}(x)$ indicates that task x consistently reflects the quality ranking of different harnesses, where stronger harnesses achieve better performance on the task. Therefore, tasks with higher sensitivity provide more reliable signals for evaluating harness evolution. In contrast, $\mathrm{Perf}(x)$ measures the average performance across harnesses, and we define task difficulty as $1 - \mathrm{Perf}(x)$, where lower average performance indicates larger performance headroom.

Table 2: Evo-Bench harness-evolving capability leaderboard. Methods are ranked by Overall score; the two baseline harnesses are excluded from ranking. **Bold** and underlined denote the best and second-best raw scores for each metric, respectively. Green and red values indicate absolute score changes relative to the CodeAct baseline; gray indicates no change.

Rank	Model	Search score	Office score	General score	Overall score	AnytimeVal
	Frontier models					
1	GPT-5.6 Sol OpenAI (2026)	44.5 +32.8	41.6 +3.2	**59.4** +11.0	**46.3** +16.6	50.1
2	Claude Opus 4.8 Anthropic (2026b)	**46.5** +34.8	39.7 +1.3	56.3 +7.9	45.8 +16.1	**51.4**
3	GLM-5.2 Z.ai (2026)	45.4 +33.7	39.2 +0.8	48.4 ±0.0	43.5 +13.8	51.0
4	Qwen3.7-Max Qwen Team (2026b)	36.3 +24.6	37.8 -0.6	**59.4** +11.0	41.5 +11.8	49.3
5	Minimax-M3 Lai et al. (2026)	33.6 +21.9	**41.7** +3.3	56.3 +7.9	41.4 +11.7	49.0
7	Deepseek V4 Pro DeepSeek-AI (2026)	34.4 +22.7	39.1 +0.7	48.4 ±0.0	39.1 +9.4	45.4
8	Kimi K2.7 Code Kimi AI (2026)	34.5 +22.8	38.1 -0.3	48.4 ±0.0	38.7 +9.0	43.4
	Open-weight models					
6	Qwen3.6-27b Qwen Team (2026a)	34.8 +23.1	38.8 +0.4	50.0 +1.6	39.4 +9.7	46.9
9	Gemma-4-31B Team et al. (2026)	24.2 +12.5	40.4 +2.0	50.0 +1.6	35.9 +6.2	36.2
	Baseline variants					
–	CodeAct	11.7	38.4	48.4	29.7	–
–	Artificial Harness	46.7	43.9	56.3	47.5	–

Task Suite Construction. We first remove tasks with non-positive $\mathrm{Sens}(x)$, as they provide limited signal for distinguishing harness quality. Among the remaining tasks, we partition them into difficulty strata according to $1 - \mathrm{Perf}(x)$ and select tasks with the highest $\mathrm{Sens}(x)$ within each stratum. This procedure yields a task set that preserves difficulty diversity while prioritizing tasks that are more responsive to harness evolution. Then we randomly split tasks within each stratum into validation and evaluation suites, ensuring that they follow the same difficulty distribution. Table 1 reports the intermediate statistics of the task selection process.

5 Experiments

In this section, we present the main results of Evo-Bench. We evaluate the harness-evolving capabilities of frontier models, analyze their evolve process, and further discuss under what conditions model-evolved harnesses can surpass the current state-of-the-art human-engineered frameworks.

5.1 Experimental Setup

Baselines. We compare the evolved harnesses against two baselines: (1) the initial **CodeAct** seed harness, which serves as the starting point to quantify absolute evolutionary gains, and (2) **Artificial harness**, a composite of domain-specific human-engineered frameworks: MiroFlow Team et al. (2025) for search, Stirrup Artificial Analysis, Inc. (2026) for office, and Claw-Eval Ye et al. (2026) for general tasks.

5.1.1 Evaluation protocol.

Unless stated otherwise, all experiments use DeepSeek-V4-Flash as the fixed policy model and start from the same CodeAct seed policy harness H_0. The main experiments use a common budget of 20 iterations, 1,000 evolver steps, and 48 hours. Each policy rollout is capped at 300 steps and one hour. Search and office tasks use one rollout, whereas Claw-Eval use three rollouts, following its native three-trial $\mathrm{Pass}\hat{}3$ metric. Tasks requiring LLM-based grading use Qwen3.7-Plus as the common judge. We run all experiments one time, and report domain-level scores, Overall Score, and Anytime Validation Score (AnytimeVal), as defined in the previous section. Costs are computed from recorded input, cached-input, and output tokens using provider prices as of July 10, 2026.

5.1.2 Model configuration.

We evaluate seven frontier models and two open-weight models on Evo-Bench across the following settings. All evolvers run in thinking mode with temperature 1.0 and the largest context window

supported by the model. For models exposing a reasoning-effort control, we use their maximum setting; for models that do not expose a configurable reasoning-effort level, we use their default setting. The shared DeepSeek-V4-Flash policy model uses maximum reasoning effort, temperature 1.0, and a 256K-token context window. The Qwen3.7-Plus judge uses temperature 0.0 and a 1M-token context window.

5.2 Main Results

Overall performance. Table 2 presents the overall leaderboard of all evaluated models. GPT-5.6-Sol and Claude Opus-4.8 lead with scores of 46.3 and 45.8 respectively, yielding massive absolute gains over the initial CodeAct baseline. This widespread positive delta validates that frontier LLMs possess a genuine capability to autonomously optimize executable harnesses. Examining evolutionary progress, Claude Opus-4.8 and GLM-5.2 achieve the highest Anytime Validation scores. This early peak suggests they rapidly evolve high-quality structures but frequently introduce detrimental modifications in subsequent iterations. Despite these gains, the top evolved harness still slightly underperforms domain-specific Artificial harness composite score of 47.5, leaving headroom for future research.

Per-domain performance. The evolutionary gains are highly uneven across different task domains. **Search** tasks are more amenable to optimization, with Claude Opus-4.8 gaining +34.8 to nearly match the Artificial harness, indicating evolvers easily synthesize missing web-navigation logic. Conversely, **Office** tasks remain stubborn. Most models show marginal improvements or slight regressions, failing to match the Artificial baseline likely due to the difficulty of discovering specialized workflows. Interestingly, for **General** tasks, top evolvers like GPT-5.6-Sol and Qwen3.7-Max strictly surpass the Artificial harness. This milestone demonstrates that autonomously evolved reasoning structures can indeed outstrip manually crafted solutions.

Budget use. Figure 4 reveals two distinct evolutionary behaviors: exhaustive exploration and early saturation. GPT-5.6-Sol and Kimi-K2.7-Code uniquely exhaust the maximum 20-iteration budget with the highest step counts and longest durations. This persistent exploration directly contributes to the top-tier overall performance of GPT-5.6-Sol. Conversely, most models terminate prematurely due to invalid code proposals or stagnant reasoning loops. For instance, Qwen3.7-Max and DeepSeek-V4-Pro halt at 15 iterations using barely 200 steps. However, budget exhaustion does not strictly dictate success. Despite its early halt, Qwen3.7-Max delivers a highly competitive score and dominates the General tasks domain, demonstrating exceptional sample efficiency in synthesizing effective updates without exhaustive trial and error.

Cost analysis. Figure 5 illustrates the overall performance against the API inference cost incurred exclusively by the evolver model. The Pareto frontier reveals a steep logarithmic trade-off between financial investment and evolutionary capability. Achieving top-tier results demands substantial resources: GPT-5.6-Sol secures the highest score but dominates the cost spectrum by exceeding 500 USD per run. In stark contrast, models occupying the middle ground demonstrate exceptional cost-effectiveness. GLM-5.2 and Qwen3.7-Max establish the highly efficient knee of the curve, delivering formidable performance for under 40 USD. At the extreme budget end, DeepSeek-V4-Pro anchors the absolute baseline, successfully synthesizing functional harness improvements for less than a single dollar.

5.3 Further Analysis: When Can Evolved Harnesses Surpass Human-Engineered Systems?

Case study: how GPT-5.6-Sol evolves a harness. Figure 6 shows GPT-5.6-Sol acting as an autonomous systems engineer: instead of applying a uniform harness, it constructs a hierarchical router for domain-specific prompts and tools. It further demonstrates tool construction, failure diagnosis, and iterative correction. For Search, it implements web-search/fetch tools and a webpage cleaner that removes scripts, styles, and markup noise while preserving links. For Office, it separates APEX and GDPval: APEX tracks evidence by file, page, and sheet cell and recomputes key values, whereas GDPval requires artifact creation, recalculation, reopening, and rendered inspection. For General

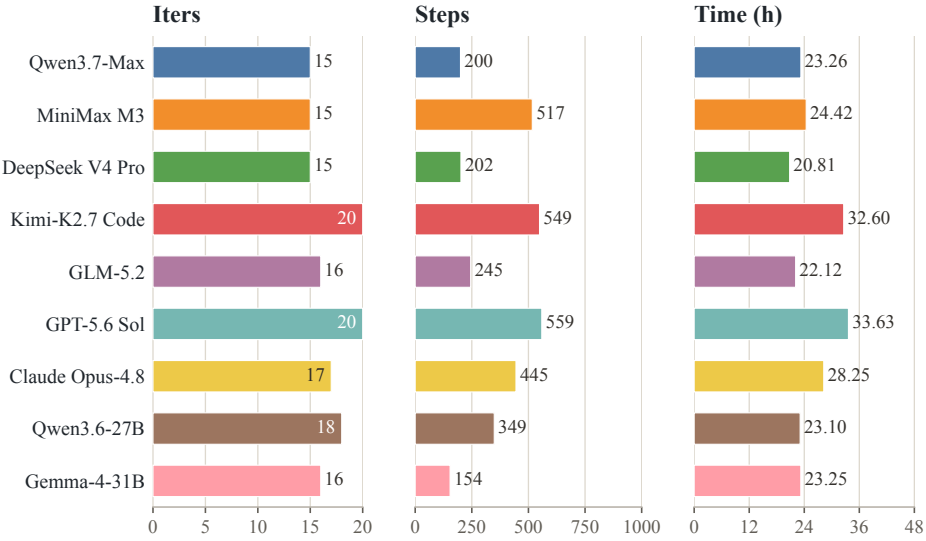


Figure 4: Budget usage of each evolver. Iters, steps and time report the number used relative to the maximum budgets of 20 iterations, 1,000 steps, and 48 hours, respectively.

tasks, it adds recovery from empty or premature responses, credential redaction, and a guard against sending draft-only emails. Across evolvers, Opus-4.8 exhibits the closest systems-engineering profile to GPT-5.6-Sol, combining tool construction with domain-aware control, context management, failure recovery, and evidence-based reversion. GLM-5.2 demonstrates substantial diagnosis and tool engineering but relies more on domain-specific guidance and local controls, while Qwen3.7-Max and MiniMax M3 focus primarily on web tools, prompt specialization, and safety filters.

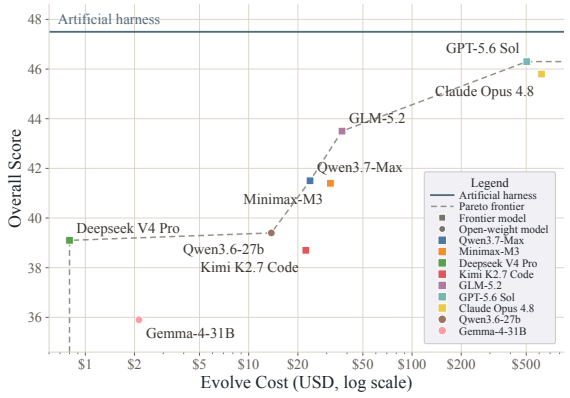


Figure 5: Cost($) vs. performance. The dashed line denotes the Pareto frontier; the horizontal line denotes the artificial harness.

GPT-5.6-Sol's Harness Evolution on Evo-Bench

```
[Steps 1-79, 199-205: Diagnose and route]
>> run_subagent(Search / Office / Claw failures)
aggregate task.domain -> Search / Office / Claw
ToolRouter: web only for Search; PromptRouter: by domain
sub-route Office task_id -> APEX / GDPval policy

[Search | Steps 5-69, 180-254: Add capability]
file_write web tools: Search 6.3 -> 18.8; violations 43
>> query_rollouts(iteration_001, violations)
file_edit HTML cleaner: remove scripts/markup noise
preserve links; stalls 4 -> 0; safe Search 45.3

[Office | Steps 74-227: Split task contracts]
APEX: source ledger + recompute | GDPval: reopen QA
Office 36.5 -> 40.0 (GDPval 67.6; APEX 12.5)
strict gate: missing files 10 -> 13 [REFUTED]; revert

[General/Claw | Steps 19-179: Add execution controls]
recover empty/premature responses; redact credentials
block draft-only sends; prefer injected app tools
Claw 50.0 -> 62.5; individual pass 64.6 -> 69.8

[Uncertainty checks and freeze | Steps 283-559]
>> get_research_state: iter. 1 -> 13/20; step 31 -> 266
stability testing: step 282 -> 559
freeze hierarchical router + supported recovery changes
Held-out: Search 44.5 < 46.7 | Office 41.6 < 43.9
General 59.4 > 56.3 | Overall 46.3 < 47.5 Official
```

Figure 6: GPT-5.6-Sol harness-evolution case study.

Challenges. Despite its achievements, GPT-5.6-Sol exposes three critical limitations. First, it reacts superficially to aggregate scores rather than distilling causal failure modes from extensive logs. Second, it resolves cross-domain interference via naive domain routing instead of discovering fundamentally robust, shared mechanisms. Third, it underutilizes the research budget, lacking the goal-oriented persistence of human engineers. Consequently, the evolved harness relies on localized modifications, leaving core components primitive: the planner acts passively, the context remains append-only, and the verifier stays entirely permissive. These behavioral patterns extend across other evolvers. Opus-4.8 exhibits the closest systems-engineering profile to GPT-5.6-Sol. In con-

trast, GLM-5.2 relies more heavily on local controls, while Qwen3.7-Max and MiniMax M3 focus primarily on basic web tools and prompt specialization.

Discussion. In summary, current frontier models demonstrate highly promising autonomous engineering capabilities, notably outperforming manual designs on General tasks. However, the overall lack of deep system-level optimization explains why they still trail the Artificial harness on Search with a score of 44.5 against 46.7, and on Office with 41.6 against 43.9. We believe that by strengthening the capacity to abstract causal failure patterns, execute large-scale architectural refactoring, and maximize budgets utilization, future language models hold substantial potential to continuously evolve harnesses that consistently surpass human experts.

6 Ablation Studies

To better understand the scalability and robustness of harness evolution, we conduct ablation studies examining two critical factors. First, we investigate how expanding the available resource budget impacts final performance. Second, we analyze whether the evolved harnesses successfully generalize across different underlying policy models.

6.1 Effect of Evolution Budget

To evaluate the scaling behavior of harness evolution, we track the performance of Qwen3.7-Max and GLM-5.2 across three different budget constraints: 24 hours, 10 iterations with 500 steps; 36 hours, 15 iterations with 750 steps; and 48 hours, 20 iterations with 1000 steps. As illustrated in Figure 7, both models demonstrate a clear positive scaling trend. Expanding the budget from 24 to 48 hours yields consistent, monotonic improvements in both the Overall Score and the Anytime Validation score.

Notably, GLM-5.2 exhibits a sharp performance trajectory, achieving a steep climb in its Anytime Validation metric during the initial expansion to 36 hours before plateauing. Conversely, Qwen3.7-Max demonstrates a steadier, linear growth across all measured constraints. This positive scaling trend indicates that despite the previously noted limitations in budget utilization, allocating greater computational resources for iterative exploration can still provide meaningful performance dividends, ultimately enabling the synthesis of more refined agent harnesses.

6.2 Effect of the Policy Model

Our main experiments fix DeepSeek-V4-Flash as the policy model. To verify whether the harness evolution capability is tied to a specific policy model, we swap it to Qwen3.6-35B-A3B and GLM-5.2, and re-evaluate the evolution process using Qwen3.7-Max and GLM-5.2 as evolvers.

Table 3 demonstrates that harness evolution is highly robust to policy model changes. Regardless of whether the policy agent is based on Qwen, DeepSeek, or GLM, the evolved harnesses consistently achieve massive improvements over their respective CodeAct baselines. For instance, when utilizing Qwen3.6-35B-A3B as the policy model, the initial Overall score starts at a modest 13.9. Yet, both evolvers successfully optimize the harness to reach scores of 27.9 and 29.2. A similar trend emerges with the highly capable GLM-5.2 policy model, where the GLM-5.2 evolver lifts the baseline score from 38.0 to an impressive 48.4. These consistent cross-policy gains demonstrate that the evolvers are genuinely synthesizing generalizable reasoning and tool-use structures rather than merely overfitting to the idiosyncratic flaws of a single target model.

7 Conclusion

In this work, we present Evo-Bench, a novel benchmark designed to evaluate language models' intrinsic capacity to autonomously improve agent harnesses. By establishing a rigorous framework anchored in controlled attribution, long-horizon iteration, and transfer alignment, Evo-Bench disentangles genuine harness-evolving capability from trivial prompt tuning and stochastic runtime noise. Our extensive evaluations across flagship LLMs reveal both the potential and current limitations of models operating as autonomous research engineers. We hope Evo-Bench serves as a foundational

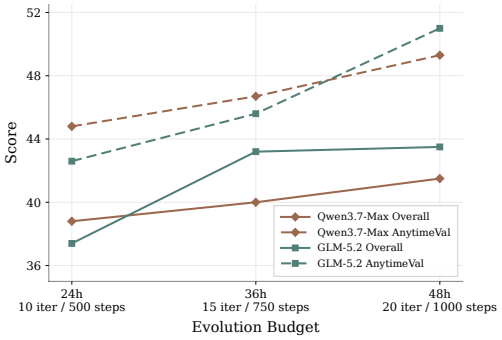


Figure 7: Effect of evolution budget.

Evolver	Search	Office	General	Overall	ATV
Policy: Qwen3.6-35B-A3B					
Baseline	2.7	14.2	35.9	13.9	–
Qwen3.7-Max	12.5	33.0	**48.4**	27.9	29.2
GLM-5.2	**16.4**	**34.0**	45.3	**29.2**	**30.8**
Policy: DeepSeek-V4-Flash					
Baseline	11.7	38.4	48.4	29.7	–
Qwen3.7-Max	36.3	37.8	**59.4**	41.5	49.3
GLM-5.2	**45.4**	**39.2**	48.4	**43.5**	**51.0**
Policy: GLM-5.2					
Baseline	18.0	40.2	73.4	38.0	–
Qwen3.7-Max	**38.3**	45.1	46.9	42.7	46.7
GLM-5.2	35.6	**45.2**	**80.3**	**48.4**	**50.4**

Table 3: Effect of the policy model on harness evolution. ATV denotes AnytimeVal; baseline is the CodeAct loop.

testbed to catalyze future research on self-evolving agent architectures and execution-grounded reasoning.

8 Future Work

Our goal is to maintain Evo-Bench as a living benchmark that continues to provide an effective measure of harness evolution—an initial, practically measurable form of AI self-evolution. In the future, we will integrate coding tasks and challenging scientific research tasks into Evo-Bench, extend compatibility across diverse agent frameworks, and evaluate a wider range of models.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. Gepa: Reflective prompt evolution can outperform reinforcement learning. In *First Workshop on Foundations of Reasoning in Language Models*, 2025.

Anthropic. Claude Code. https://www.anthropic.com/news, 2025. Research preview announced Feb. 24, 2025; general availability May 22, 2025.

Anthropic. Introducing Claude Fable 5. https://www.anthropic.com/news/claude-fable-5-mythos-5, 2026a.

Anthropic. Introducing Claude Opus 4.8. https://www.anthropic.com/news/claude-opus-4-8, 2026b.

Anthropic. Introducing claude sonnet 5. https://www.anthropic.com/news/claude-sonnet-5, 2026c.

Artificial Analysis, Inc. Stirrup: The lightweight foundation for building agents. https://github.com/ArtificialAnalysis/Stirrup, 2026.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pp. 50466–50494, 2025.

Tingyang Chen, Shuo Lu, Kang Zhao, Weicheng Meng, Hanlin Teng, Tianhao Li, Chao Li, Xule Liu, Jian Liang, Zhizhong Zhang, et al. Harnessx: A composable, adaptive, and evolvable agent harness foundry. *arXiv preprint arXiv:2606.14249*, 2026.

Zheng Chu, Xiao Wang, Jack Hong, Huiming Fan, Yuqi Huang, Yue Yang, Guohai Xu, Shengchao Hu, Dongdong Kuang, Chenxiao Zhao, Cheng Xiang, Ming Liu, Bing Qin, and Xing Yu. Redsearcher: A scalable and cost-efficient framework for long-horizon search agents. *arXiv preprint arXiv:2602.14234*, 2026. URL https://arxiv.org/pdf/2602.14234.

DeepSeek-AI. Deepseek-v4: Towards highly efficient million-token context intelligence, 2026.

Chrisantha Fernando, Dylan Sunil Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Forty-first International Conference on Machine Learning*, 2024.

Xingze Gao, Chuanrui Hu, Hongda Chen, Pengfei Yao, Zhao Wang, Yi Bai, Zhengwei Wu, Yunyun Han, Xiaofeng Cong, Jie Gui, et al. Evoagentbench: Benchmarking agent self-evolution via ability transfer. *arXiv preprint arXiv:2607.05202*, 2026.

Prannay Hebbar, Yogendra Manawat, Samuel Verboomen, Alesia Ivanova, Selvam Palanimalai, Kunal Bhatia, and Vignesh Baskaran. Sia: Self improving ai with harness & weight updates. *arXiv preprint arXiv:2605.27276*, 2026.

Dan Hendrycks, Mantas Mazeika, Oliver Zhang, Jason Hausenloy, Richard Ren, Ryan Kim, Adam Khoja, Nathaniel Li, Alice Gatti, Long Phan, et al. A benchmark of expert-level academic questions to assess ai capabilities. *Nature*, 649(8099), 2026.

Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.

Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. Mlagentbench: Evaluating language agents on machine learning experimentation. *arXiv preprint arXiv:2310.03302*, 2023.

Sihang Jiang, Lipeng Ma, Zhonghua Hong, Keyi Wang, Zhiyu Lu, Tengfei Wang, Shisong Chen, Jinghao Zhang, Tianjun Pan, Weijia Li, et al. Sea-eval: A benchmark for evaluating self-evolving agents beyond episodic assessment. *arXiv preprint arXiv:2604.08988*, 2026.

Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik R Narasimhan. SWE-bench: Can language models resolve real-world github issues? In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

Kimi AI. Kimi k2.7 code: An open-source, coding-focused agentic model built for long-horizon software engineering., 2026. URL https://www.kimi.com/resources/kimi-k2-7-code.

Xunhao Lai, Weiqi Xu, Yufeng Yang, Qiaorui Chen, Yang Xu, Lunbin Zeng, Xiaolong Li, Haohai Sun, Haichao Zhu, Vito Zhang, et al. Minimax sparse attention. *arXiv preprint arXiv:2606.13392*, 2026.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. Meta-harness: End-to-end optimization of model harnesses. *arXiv preprint arXiv:2603.28052*, 2026.

Xirui Li, Ming Li, Ion Stoica, Cho-Jui Hsieh, and Tianyi Zhou. Clawenvkit: Automatic environment generation for claw-like agents. *arXiv preprint arXiv:2604.18543*, 2026.

Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, et al. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving llm agents. *arXiv preprint arXiv:2605.30621*, 2026.

Xinyu Lu, Tianshu Wang, Pengbo Wang, Zhiqiang Zhang, Jun Zhou, Boxi Cao, Yaojie Lu, Hongyu Lin, Xianpei Han, Le Sun, et al. The meta-agent challenge: Are current agents capable of autonomous agent development? *arXiv preprint arXiv:2606.04455*, 2026.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco JR Ruiz, Abbas Mehrabian, et al. Alphaevolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.

OpenAI. Codex. https://openai.com/index/introducing-codex/, 2025. Terminal CLI announced Apr. 16, 2025; cloud-agent research preview announced May 16, 2025.

OpenAI. Gpt-5.6: Frontier intelligence that scales with your ambition. https://openai.com/index/gpt-5-6/, 2026.

Tejal Patwardhan, Rachel Dias, Elizabeth Proehl, Grace Kim, Michele Wang, Olivia Watkins, Simón Posada Fishman, Marwan Aljubeh, Phoebe Thacker, Laurance Fauconnet, et al. Gdpval: Evaluating ai model performance on real-world economically valuable tasks. *arXiv preprint arXiv:2510.04374*, 2025.

Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026a. URL https://qwen.ai/blog?id=qwen3.6-27b.

Qwen Team. Qwen3.7: The agent frontier, May 2026b. URL https://qwen.ai/blog?id=qwen3.7.

Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. Posttrainbench: Can llm agents automate llm post-training? *arXiv preprint arXiv:2603.08640*, 2026.

Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias, Evan Mays, Benjamin Kinsella, Wyatt Thompson, et al. Paperbench: Evaluating ai's ability to replicate ai research. *arXiv preprint arXiv:2504.01848*, 2025.

Gemma Team, Sherif El Abd, Vaibhav Aggarwal, Robin Algayres, Alek Andreev, Olivier Bachem, Ian Ballantyne, Cormac Brick, Victor Cărbune, Michelle Casbon, et al. Gemma 4 technical report. *arXiv preprint arXiv:2607.02770*, 2026.

MiroMind Team, Song Bai, Lidong Bing, Carson Chen, Guanzheng Chen, Yuntao Chen, Zhe Chen, Ziyi Chen, Xuan Dong, et al. Mirothinker: Pushing the performance boundaries of open-source research agents via model, context, and interactive scaling. *arXiv preprint arXiv:2511.11793*, 2025.

MiroMind Foundation Model Team and MiroMind AI Infra Team. Mirorl: An mcp-first reinforcement learning framework for deep research agent. https://github.com/MiroMindAI/MiroRL, 2025.

Varun Ursekar, Apaar Shanker, Veronica Chatrath, Yuan Xue, and Samuel Marc Denton. Vero: A harness for agents to optimize agents. *arXiv preprint arXiv:2602.22480*, 2026.

Bertie Vidgen, Austin Mann, Abby Fennelly, John Wright Stanly, Lucas Rothman, Marco Burstein, Julien Benchek, David Ostrofsky, Anirudh Ravichandran, Debnil Sur, et al. Apex-agents. *arXiv preprint arXiv:2601.14242*, 2026.

Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. Executable code actions elicit better llm agents. In *Forty-first International Conference on Machine Learning*, 2024.

Yike Wang, Huaisheng Zhu, Zhengyu Hu, Yige Yuan, Zhengyu Chen, Shakti Senthil, Hannaneh Hajishirzi, Yulia Tsvetkov, Pradeep Dasigi, and Teng Xiao. Rethinking the evaluation of harness evolution for agents. *arXiv preprint arXiv:2607.12227*, 2026.

Jason Wei, Zhiqing Sun, Spencer Papay, Scott McKinney, Jeffrey Han, Isa Fulford, Hyung Won Chung, Alex Tachard Passos, William Fedus, and Amelia Glaese. Browsecomp: A simple yet challenging benchmark for browsing agents. *arXiv preprint arXiv:2504.12516*, 2025.

Lilian Weng. Harness engineering for self-improvement. *Lil'Log*, July 2026. URL https://lilianweng.github.io/posts/2026-07-04-harness/.

Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. Re-bench: Evaluating frontier ai r&d capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh J Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, et al. Osworld: Benchmarking multimodal agents for open-ended tasks in real computer environments. *Advances in Neural Information Processing Systems*, 37:52040–52094, 2024.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. *arXiv e-prints*, art. arXiv:2406.12045, June 2024. doi: 10.48550/arXiv.2406.12045.

Bowen Ye, Rang Li, Qibin Yang, Yuanxin Liu, Linli Yao, Hanglong Lv, Zhihui Xie, Chenxin An, Lei Li, Lingpeng Kong, et al. Claw-eval: Towards trustworthy evaluation of autonomous agents. *arXiv preprint arXiv:2604.06132*, 2026.

Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement, 2024. URL https://api.semanticscholar.org/CorpusID:273186154.

Z.ai. Glm-5.2: Built for long-horizon tasks. https://z.ai/blog/glm-5.2, 2026.

Eric Zelikman, Eliana Lorch, Lester Mackey, and Adam Tauman Kalai. Self-taught optimizer (stop): Recursively self-improving code generation. In *First conference on language modeling*, 2024.

Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, et al. Glm-5: from vibe coding to agentic engineering. *arXiv preprint arXiv:2602.15763*, 2026.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026a.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Tjarko Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations*, 2026b. URL https://openreview.net/forum?id=pUpzQZTvGY.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xiong-Hui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations*, 2025a. URL https://openreview.net/forum?id=z5uVAKwmjf.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025b.

Congjie Zheng, Chuanyi Xue, Bin Liang, Jun Yang, and Changshui Zhang. Seagym: An evaluation environment for self-evolving llm agents. *arXiv preprint arXiv:2606.17546*, 2026.

Appendix

A Benchmark Details **16**

A.1 Details of the Evolve Harness 16

A.2 Details of the Policy Harness 17

A.3 Benchmark Difficulty and Harness Sensitivity 17

B Details of Benchmark Construction **17**

B.1 Selection of Auxiliary Tasks 18

B.2 Selection of Auxiliary Harnesses 18

C Experiment Details **19**

C.1 Model Configurations and Cost Accounting 19

C.2 Integrity and Reward-Hacking Analysis 20

D Failure Mode Analysis of Evolution Trajectories **20**

D.1 Qwen3.6-27B: Misattributing Regressions to Noise 22

D.2 DeepSeek-V4-Pro: Premature Plateau and Procedural Evaluations 22

D.3 Kimi-K2.7-Code: Careful Rollback but Local Search Saturation 22

D.4 Cross-Model Failure Patterns 22

E Prompt Design **22**

E.1 Evolver Prompt 22

E.2 Seed Policy Prompt 23

E.3 Judge Prompts 24

A Benchmark Details

This section supplements the benchmark description with implementation and diagnostic details. It describes the fixed evolve harness, the seed policy harness, and the task-level difficulty and harness-sensitivity characteristics of the constructed benchmark suites.

A.1 Details of the Evolve Harness

The fixed evolve harness $\mathcal{H}_{\text{evo}}$ lets every evolver inspect, revise, and evaluate a policy harness under the same prompt, tools, state management, and resource accounting. Table 4 summarizes its components.

Table 4: Components of the evolve harness.

Component	Function
Evolution orchestrator	Defines the optimization objective, hypothesis-driven workflow, editable surfaces, integrity rules, and termination protocol.
Engineering tools	Provide file inspection and editing, code search, shell execution, web access, and read-only delegated exploration.
Research skills	Support rollout slicing, behavioral comparison, multi-eval analysis, experiment logging, insight retention, and architecture-level reflection.
Evaluation interface	Evaluates immutable harness snapshots asynchronously and returns scores, per-domain results, task outcomes, and failure diagnostics.
Context and state management	Tracks budgets, evaluations, best revisions, and a persistent experiment ledger, while compacting old interaction history for long runs.
Sandbox and snapshot management	Separates evolver evidence from policy inputs, records revisions and trajectories, and freezes one harness for held-out evaluation.

Evolution orchestrator. The orchestrator separates the editable policy harness from a scratch workbench and promotes a diagnose–hypothesize–edit–evaluate cycle over all harness surfaces. A live state view reports evaluation, step, time, and token budgets, recent scores, and experiment history; limits are enforced by the harness.

Engineering tools. File, search, shell, and web tools support implementation and external research. A read-only subagent can inspect code, validation artifacts, and web sources in an independent context, but cannot edit or evaluate the harness.

Research skills. Six progressively disclosed skills provide rollout filtering (`query_rollouts`), task-paired comparison (`diff_task`), and multi-evaluation analysis (`analyze_evals`). The ledger skills record hypotheses (`log_experiment`), close them from evidence (`record_insight`), and trigger broader redesign after repeated local failures (`architecture_checkpoint`). All are optional views over raw, evolver-visible artifacts.

Evaluation interface. `run_train_eval` consumes one iteration and asynchronously evaluates an immutable snapshot; at most one evaluation is active. `await_eval` returns the headline metric, changes from previous and best revisions, per-domain results, worst tasks, and failure classes. Full results, feedback, logs, and trajectories remain available.

Context and state management. The harness records interactions, tool calls, evaluations, resource use, and snapshots; the ledger persists resolved hypotheses. Near the context limit, older messages are summarized while recent turns and all disk artifacts remain available.

Sandbox and snapshot management. The evolver may read validation evidence but write only the policy harness and workbench. Policy rollouts receive a read-only snapshot, public task, and isolated workspace, never answers, scorers, evolver files, or held-out data. Integrity checks scan source and trajectories; held-out evaluation begins only after the selected revision is frozen.

A.2 Details of the Policy Harness

All runs start from the CodeAct seed policy harness H_0 Wang et al. (2024) in Figure 8. The fixed policy model alternates shell commands and append-only observations until `finish`, a tool-used answer, or a resource limit. Domain tools, planning, memory, and verification are left for evolution.

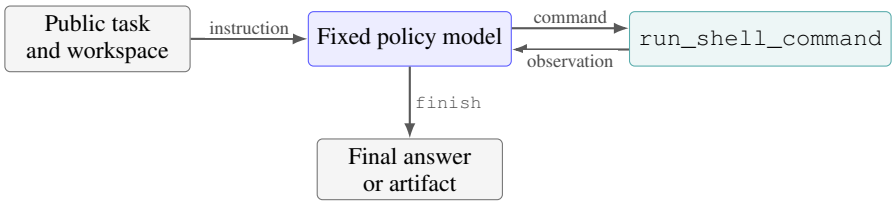


Figure 8: The minimal CodeAct policy harness. Shell observations are appended to the model context until the task is completed or a resource limit is reached.

A.3 Benchmark Difficulty and Harness Sensitivity

Figure 9 plots harness sensitivity $\mathrm{Sens}(x)$ and difficulty $1 - \mathrm{Perf}(x)$, where $\mathrm{Perf}(x)$ is mean performance across the twelve auxiliary harnesses. Selection favors non-negative sensitivity while preserving difficulty coverage, and the two splits remain aligned. BrowseComp, HLE, and APEX-Agents skew hard; GDPval is broader and Claw-Eval easier. Eight slightly negative Claw-Eval tasks are quota backfills from sparse strata.

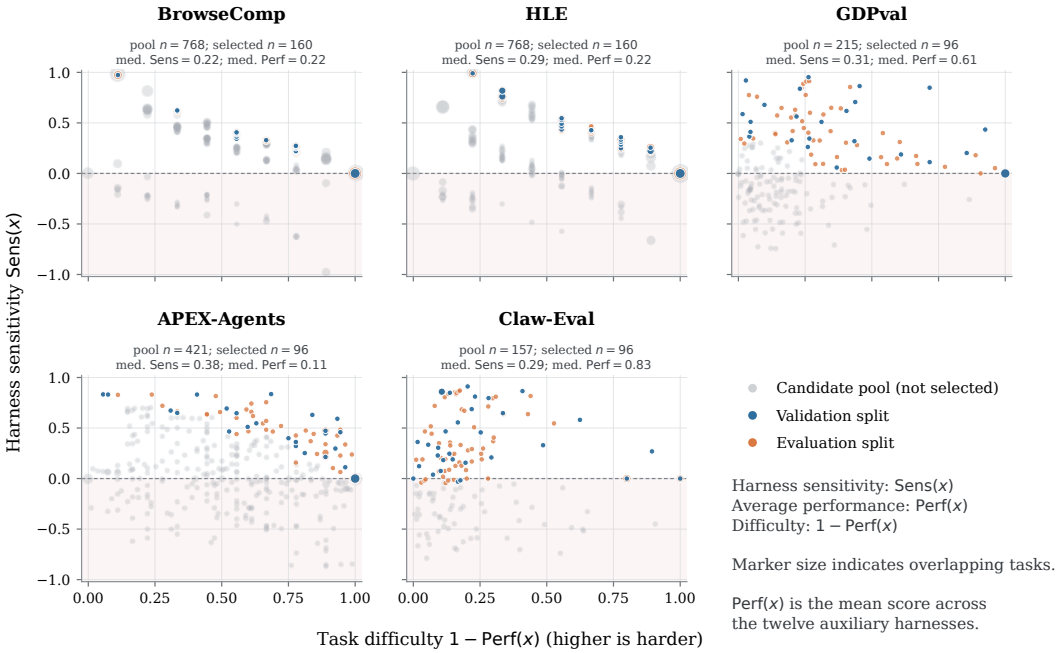


Figure 9: Difficulty and harness sensitivity of candidate and selected benchmark tasks. Gray, blue, and orange denote unselected, validation, and evaluation tasks; marker size denotes coordinate multiplicity. Annotations give pool and selected sizes, and shading marks negative sensitivity.

B Details of Benchmark Construction

Section 4.1 summarizes the two steps of Stage 1: we first select a 320-task auxiliary suite and then use it to generate and select a representative set of auxiliary harnesses. This section provides the full selection details for both components. Section B.1 describes the selection of auxiliary tasks, and Section B.2 describes the selection of auxiliary harnesses.

B.1 Selection of Auxiliary Tasks

The first step of Stage 1 selects the auxiliary tasks used to induce harness diversity. We collect Search from MiroRL Team & Team (2025), RedSearcher Chu et al. (2026), and internal corpora; Office from anonymized enterprise workflows; and General from Auto-ClawEval Li et al. (2026). After validity, rollout-coverage, deduplication, and cross-suite leakage checks, 11,322 candidates remain: 2,988 Search, 7,564 Office, and 770 General.

Rollout-based characterization. Using DeepSeek-V4-Flash with the domain-native MiroFlow Team et al. (2025), Stirrup Artificial Analysis, Inc. (2026), and Claw-Eval Ye et al. (2026) workflows, we estimate mean score $s(x)$ and turns $\ell(x)$. We require two judged Search rollouts and three valid Office/General rollouts. This process is independent of the CodeAct seed.

Headroom–horizon selection. Within each domain, tasks enter six score bands: *frontier* ($s = 0$), *hard* ($0 < s \leq 0.10$), *hardish* ($0.10 < s \leq 0.25$), *mid* ($0.25 < s \leq 0.50$), *solvable* ($0.50 < s \leq 0.75$), and *easy* ($s > 0.75$), with quota weights $(0.58, 0.24, 0.11, 0.045, 0.02, 0.005)$. Deficits are redistributed, and candidates within each band are ranked by decreasing $\ell(x)$ with seed 20260716. This selects 128 Search, 128 Office, and 64 General tasks, favoring headroom and long interaction horizons (Figure 10).

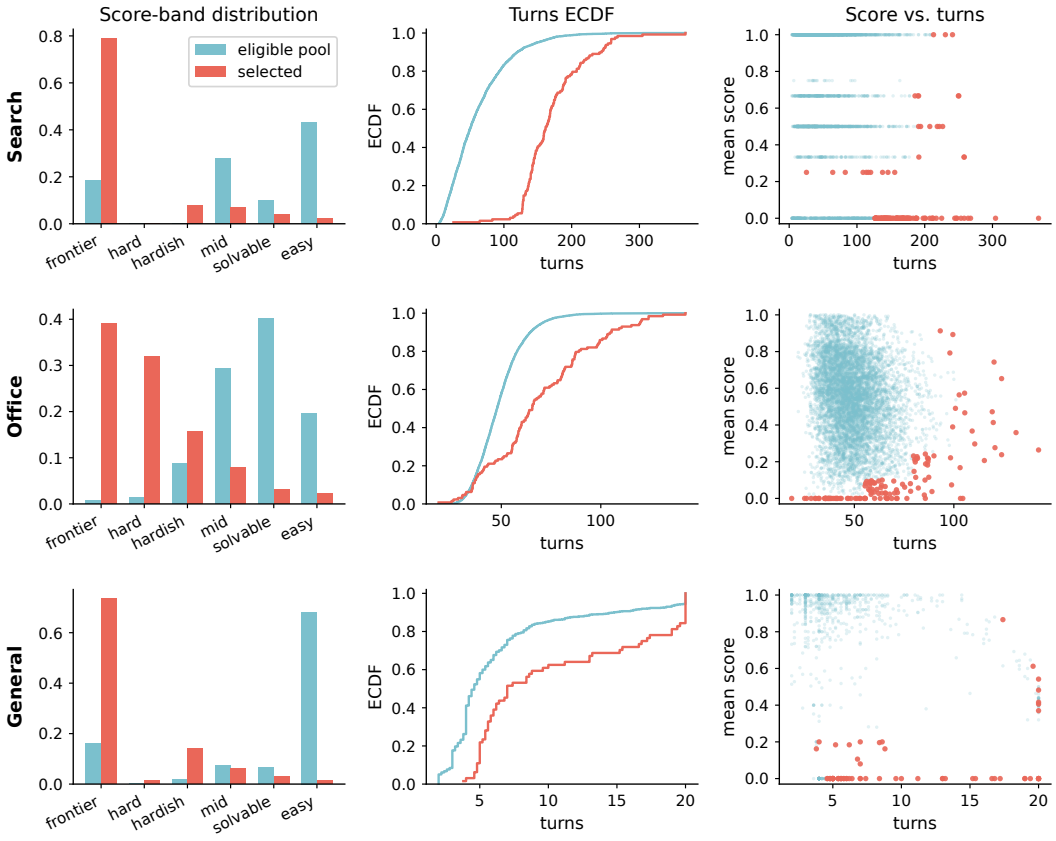


Figure 10: Auxiliary-task selection by headroom and interaction horizon. Rows are Search, Office, and General; columns show score bands, turn CDFs, and score–turn distributions. Blue is the candidate pool and red the selected 320-task suite.

B.2 Selection of Auxiliary Harnesses

After selecting the auxiliary tasks, the second step of Stage 1 uses them to generate a diverse pool of harness revisions and select a representative auxiliary harness set. Claude Opus-4.8, Claude

Sonnet-5, GLM-5.2, and GPT-5.6-Sol independently evolve the same seed CodeAct harness on the 320 auxiliary tasks with DeepSeek-V4-Flash as the fixed policy model and the main budget (20 iterations, 1,000 steps, 48 hours). Holding the starting harness, policy model, evolution protocol, and resource budget constant isolates variation induced by the four evolvers. Together, the four evolutions produce 73 evaluated harnesses for the subsequent selection procedure.

Snapshot deduplication and representation. Because successive iterations can produce code-identical harnesses, we first deduplicate the candidate pool. Content hashing reduces the 73 evaluated harnesses to 65 unique revisions; the common seed harness is excluded. To compare the remaining revisions on a common basis, we represent each one by its exercised and coded capabilities, observed tool use, program structure, and auxiliary performance. The capability representation covers tooling, context, control flow, verification, memory, recovery, multi-agent execution, and routing.

Representative-anchor selection. We then select a balanced subset that preserves the progression within each evolution while covering distinct harness designs. A deterministic, final-snapshot-constrained k-medoids procedure selects three harnesses per evolution. It retains the final revision, preserves capability levels seen in at least two revisions, minimizes representation error, and breaks ties by pairwise diversity. Selecting the same number of harnesses from each evolution prevents evolutions with more evaluated revisions from dominating the selected set. The four evolutions therefore yield 12 harnesses, $\mathcal{H}_{\text{aux}}$, completing the auxiliary-harness selection step of Stage 1. As described in Section 4.2, candidate-task evaluations under this set are subsequently used in Stage 2 to compute $\text{Sens}(x)$ and $\text{Perf}(x)$.

C Experiment Details

This section supplements the experimental setup and reporting protocol. It documents model configurations and cost accounting, followed by the integrity controls and trajectory audits used to detect and correct reward-hacking behavior.

C.1 Model Configurations and Cost Accounting

We evaluate seven frontier and two open-weight evolvers (Table 5) using thinking mode, the largest supported context, and the highest exposed reasoning effort. Requests allow 65,536 output tokens and time out after 600 seconds; default temperature is 1.0 unless endpoint-controlled. Policy and judge are fixed.

Table 5: Model configurations used in the main experiments. "Enabled" denotes a model-specific thinking switch rather than a scalar reasoning-effort setting. "Default" indicates endpoint-controlled temperature.

Role	Model	Reasoning	Temp.	Context
Evolver	Qwen3.7-Max	`xhigh`	1.0	1M
Evolver	MiniMax M3	`max`	1.0	1M
Evolver	DeepSeek-V4-Pro	`max`	1.0	1M
Evolver	Kimi-K2.7-Code	Enabled	Default	256K
Evolver	GLM-5.2	`max`	1.0	1M
Evolver	GPT-5.6-Sol	`max`	1.0	384K
Evolver	Claude Opus-4.8	`max`	1.0	1M
Evolver	Qwen3.6-27B	`xhigh`	1.0	256K
Evolver	Gemma 4 31B	Enabled	1.0	256K
Policy (fixed)	DeepSeek-V4-Flash	`max`	1.0	256K
Judge (fixed)	Qwen3.7-Plus	Standard	0.0	1M

Fixed policy and judge. DeepSeek-V4-Flash uses maximum reasoning, 256K context, the same output cap, and a 600-second timeout. Qwen3.7-Plus grades model-judged tasks at temperature 0.0,

1M context, the same cap, and a 1,200-second timeout. Claude Opus-4.8 uses Anthropic messages; other models use OpenAI-compatible interfaces.

Cost accounting. From per-call usage, model m costs

$$C_m = \frac{1}{10^6} \sum_r \left(n_r^{\text{in}} p_m^{\text{in}} + n_r^{\text{cr}} p_m^{\text{cr}} + n_r^{\text{cw}} p_m^{\text{cw}} + n_r^{\text{out}} p_m^{\text{out}} \right),$$

where n counts uncached input, cache-read, cache-write, and output tokens, and p is the corresponding public non-batch list price per million tokens on July 10, 2026. Request length selects tiered prices. The cost–performance analysis includes main-loop, compaction, and subagent calls but excludes policy and judge costs; full decompositions are retained. Missing cache counters are charged as uncached input.

C.2 Integrity and Reward-Hacking Analysis

Harness evolution deliberately gives the evolver broad autonomy within an independent sandbox. It can inspect rollout trajectories, task provenance, expected answers, and validation scores; use shell and web tools; and modify the policy harness. This access is necessary for diagnosing failures and developing general harness improvements, but it also creates substantial opportunities for reward hacking. During our experiments, we focused on three main forms of hacking behavior and introduced a corresponding control for each one.

(1) Leakage across the evolver–policy boundary. An evolver could misuse validation access by transferring protected information into policy execution or by attempting to interfere with the evaluation protocol. We control this risk through strict sandbox separation. The evolver may read validation artifacts but can write only to its policy harness and workbench. Each policy rollout instead receives an immutable harness snapshot, a public task view with answers and scorer fields removed, and a fresh workspace; validation artifacts, evolver notes, sibling workspaces, and held-out data are not mounted. Scorers, aggregation logic, suite definitions, and iteration accounting remain in the fixed host-side framework. Every formal validation evaluation consumes one iteration, and held-out evaluation starts only after the selected harness is frozen and returns no evidence to the evolver.

(2) Retrieval of benchmark answers. A model with web access may search for benchmark-specific answers, answer keys, or evaluation artifacts instead of solving a task from permitted evidence. Simply disabling web access is not appropriate because retrieval is intrinsic to Search tasks. We therefore scan content returned by web and shell tools for known patterns associated with benchmark datasets, answer keys, reference artifacts, and evaluation dumps. The check applies to information delivered to the policy rather than to attempted queries: an unsuccessful request is not penalized, and a request blocked before protected content reaches the policy is not treated as a violation. For a confirmed match, we record the source and tool event, mark the trial as a policy violation, and set all native score fields to zero before computing domain scores and Overall Score.

(3) Evasion of the retrieval safeguard. A model may also attempt to bypass the preceding scanner by paraphrasing, encoding, obfuscating, or concealing retrieved information. Because pattern matching alone cannot reliably detect such behavior, we apply the same Codex-based semantic audit to the complete evolver and policy trajectories of every evolution run. The audit checks whether protected information reached policy execution, whether retrieval targeted benchmark-specific artifacts, and whether behavior attempted to conceal detector-visible evidence. Among all evaluated models, only MiniMax M3 exhibited this detector-evasion behavior. The audit identified the affected trials; we set their scores to zero, recomputed the aggregates, and report the corrected MiniMax M3 result. Tool calls, observations, harness snapshots, and violation records are retained for inspection.

D Failure Mode Analysis of Evolution Trajectories

We present the complete evolution process of three lower-scoring models and analyze the failure modes that limited their final performance. The analysis draws on transcripts, experiment ledgers, validation digests, and harness snapshots. Scores are validation Overall Scores on the same 0–100 scale used in the main experiments; Ii denotes iteration i, and evaluation-suite results are excluded.

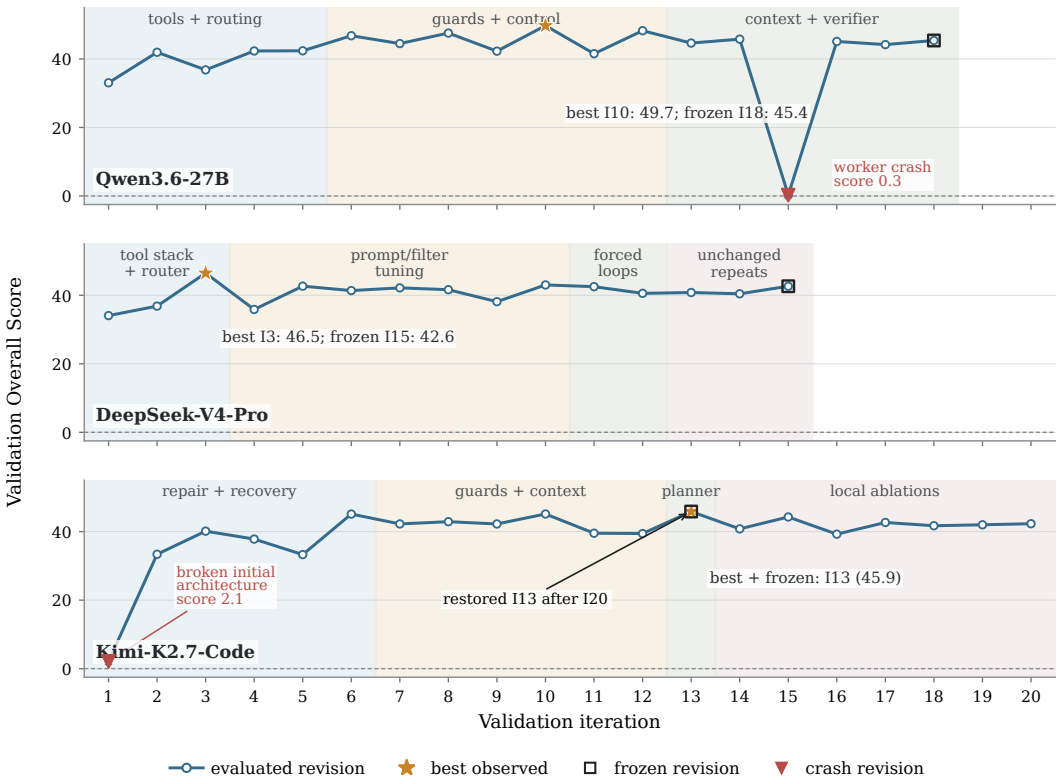


Figure 11: Validation Overall Score trajectories. Bands show intervention surfaces; stars and squares mark best and frozen revisions. Catastrophic revisions are annotated. Kimi restored I13 after I20.

Table 6: Research-budget use and revision selection. "Rollout accesses" counts raw-artifact accesses; Kimi restored the byte-identical I13 snapshot after I20.

Evolver	Iterations	Steps	Rollout accesses	Best → frozen
Qwen3.6-27B	18/20	349/1000	16	49.7 (I10) → 45.4 (I18)
DeepSeek-V4-Pro	15/20	202/1000	4	46.5 (I3) → 42.6 (I15)
Kimi-K2.7-Code	20/20	549/1000	52	45.9 (I13) → 45.9 (I13)

D.1 QWEN3.6-27B: MISATTRIBUTING REGRESSIONS TO NOISE

Qwen rapidly added retrieval, file/Python tools, routing, integrity filters, verification, and recovery, rising from 33.0 to 49.7 at I10. Later it bundled context and verifier changes; a malformed I15 verifier crashed 159/160 tasks and reduced the score to 0.3, a defect missed by local preflight. Repairs recovered only to 45.4 at I18.

Despite a 2.2-point Overall Score range across byte-identical I8/I10/I12 revisions, Qwen attributed the 4.3-point I10–I18 gap largely to noise without replicating or restoring I10/I12. It froze I18 with two iterations and 651 steps unused. The failure combines bundled edits, missing smoke checks, and weak best-snapshot management.

D.2 DEEPSEEK-V4-PRO: PREMATURE PLATEAU AND PROCEDURAL EVALUATIONS

DeepSeek reached 46.5 at I3 after adding search, fetch, file/Python tools, URL filters, and domain guidance. It then tuned prompts, filters, and completion rules while accessing raw rollouts only four times; I4–I10 ranged from 35.9 to 43.0. Requiring three searches at I11 reduced the score to 42.5 and tripled runtime; I12 reflection reached 40.6.

After an attempted early stop, I13–I15 scored 40.8, 40.4, and 42.6, with the last two evaluating unchanged code. DeepSeek labeled I3 an outlier without reevaluating it and froze I15 with five iterations and 798 steps unused. This is aggregate hill climbing without causal localization, followed by procedural budget use and premature plateau declaration.

D.3 KIMI-K2.7-CODE: CAREFUL ROLLBACK BUT LOCAL SEARCH SATURATION

Kimi recovered from an invalid I1 architecture (score 2.1) to 45.1 at I6, tested and reverted regressive context and finish controls, then reached 45.9 at I13 with a step-budget planner. Seven further tests of planning, APEX instructions, thinking, checklists, step limits, and verification did not improve it. Kimi logged these results, restored byte-identical I13, and froze the best revision.

Its limitation was search efficiency: despite exhausting all 20 evaluation iterations and accessing 52 rollouts, it used only 549/1,000 steps. Post-I13 work remained local prompt, gate, context, and threshold variants rather than deeper planning or verification redesign. Rollback discipline prevented regression but not architectural saturation.

D.4 CROSS-MODEL FAILURE PATTERNS

All three add obvious capabilities quickly but later favor local prompts, filters, thresholds, and gates over causal, task-paired redesign. Qwen and DeepSeek stop early; Kimi exhausts evaluations on low-yield search. Only Kimi restores its best snapshot. Stronger evolution therefore needs paired failure analysis, one falsifiable mechanism per iteration, cheap preflight checks, automatic best-revision recovery, and an architectural-reset trigger after repeated local failures.

E PROMPT DESIGN

This section summarizes the prompts used throughout the Evo-Bench evaluation process. They comprise a fixed evolver prompt, an editable policy prompt, and fixed evaluation-side judge prompts. Evolvers may modify only the policy layer; policies never receive references, rubrics, or judge instructions.

E.1 EVOLVER PROMPT

All nine evolvers receive the same template below, apart from serialization. Bracketed fields supply domains, metric, baseline, and budget; only repeated motivational text and expanded schemas/catalogs are omitted.

Fixed evolver prompt template

Identity and objective. You are a harness research engineer operating in an isolated sandbox. Improve the editable policy harness to maximize the held-out headline metric. Use validation only, prefer general mechanisms, and diagnose trajectories before structural changes; treat regressions as evidence.
Boundaries and integrity. The policy harness at `[POLICY]` is the product that will be frozen and scored; `[WORKBENCH]` is unscored scratch space. Validation answers, feedback, and rollouts are diagnostic evidence, not policy resources. Do not modify evaluation, transfer protected evidence, exploit artifacts, or conceal policy actions.
Tools. Use engineering tools and read-only subagents for scoped research; `get_research_state` for budgets; `run_train_eval`/`await_eval` for formal evaluation; and research skills for rollout analysis and the experiment ledger. Call `complete_evolution` only after restoring the selected revision.
Experimental workflow. Treat the clean seed as iteration 0. Repeatedly inspect failures, state a falsifiable mechanism, edit and locally test, evaluate an immutable snapshot, and retain or revert from evidence. Any harness surface may be redesigned. Analyze while evaluations run and restore the best revision before freezing.
Opening run specification. The run contains `[DOMAIN DESCRIPTIONS]` and optimizes `[HEADLINE METRIC]`. The clean baseline is `[BASELINE DIGEST]`. Redesign the minimal seed within the evaluation, step, and time budgets; evaluation-suite tasks are unavailable.
Research state. `iteration [USED]/[MAX]; step [USED]/[MAX]; [TIME] remaining; tokens [COUNT]; current/best score [VALUES]; active evaluation [STATE].`

The dataset block gives task families and 2:2:1 aggregation, but no held-out instances. Live state comes from tools rather than self-report. Near the context limit, a compaction prompt preserves per-iteration mechanisms, scores, hypotheses, best snapshot, open questions, and paths; only old messages are replaced, not disk artifacts.

E.2 Seed Policy Prompt

The minimal seed specifies only CodeAct interaction and workspace boundaries.

Seed policy system prompt

```
You are a CodeAct-style task-solving agent.

You work inside a dedicated task workspace directory.  Every
run_shell_command runs with that workspace as its working directory,
so relative paths resolve against it.  Create, edit, and read your
files there.  The concrete absolute workspace path is given in the
task message.  Use run_shell_command to inspect files, run programs,
edit files, and verify your work.  Use finish when the task is done.

Rules:
- Keep all files inside the task workspace.
- Do not access paths outside it unless explicitly shown in the task.
- Prefer simple shell commands and small file edits.
- If a command fails, inspect the output and continue.
- Call finish with a concise final answer or artifact path.
```

For task x, the seed task analyzer constructs the user message as follows:

```
Task id:  [TASK_ID]
Task workspace (your working directory):  [WORKSPACE]

[PUBLIC TASK PROMPT]

Public files:
- [WORKSPACE-RELATIVE FILE]
```

The file block is optional and the seed omits the domain label. Scorer fields, expected outputs, anchors, and rationales are stripped. Evolved prompt and code are frozen together.

E.3 JUDGE PROMPTS

Judge instructions are fixed and post-rollout. Deterministic scorers use no prompt; model judges use Section C.1 and receive references only inside evaluation (Table 7).

Table 7: Fixed judge-prompt interfaces. All structured outputs are parsed and retried on formatting failure; failed grading runs are surfaced as evaluation errors rather than silently treated as valid model failures.

Scorer	**Judge evidence**	**Required decision**
BrowseComp / semantic equivalence	Question, protected short reference answer, and policy final answer	`CORRECT: YES/NO` plus one-sentence reason; formatting and equivalent phrasing are ignored.
HLE	Question, policy response, and precise protected answer	Extract the final answer, judge strict equivalence (with a small numerical tolerance), and return reasoning, a yes/no verdict, and confidence.
GDPval absolute	Task, weighted positive/penalty criteria, parsed deliverable text, formulas, and rendered pages/slides	JSON with points and evidence-backed reason for every criterion, plus an overall summary.
GDPval pairwise	The same rubric plus policy and frozen-seed deliverables, including text and images	For every criterion choose A, B, or tie. Both A/B orders are judged and averaged to reduce position bias.
Claw-Eval	Complete executed tool-use trace, task-private grader, rubric, and reference state	Benchmark-native completion, robustness, communication, and safety scores, combined by the official grader.
APEX-Agents	Policy trajectory, environment-state difference, artifacts, and task-private verifiers inside the APEX sandbox	Per-verifier rationale and score; a task passes only when every required criterion passes.

Semantic grading ignores surface form; HLE uses extract-then-compare. GDPval uses rendered images for visual criteria and parsed text for facts/formulas; pairwise grading averages both A/B orders. APEX-Agents and Claw-Eval retain native graders. Validation rationales are evolver-visible but policy-hidden; held-out feedback is generated only after freezing.